

Chapter 2 Operating-System Structures

Outline

- Operating System Services
- User and Operating System-Interface
- System calls
- System services
- Linkers and Loaders
- Why applications are operating system Specific
- *Design and implementation*
- Operating system structure
- *Building and booting an operating system*
- *Operating system debugging*

Objectives

- Identify services provided by an operating system
- Illustrate how system calls are used to provide operating system services
- Compare and contrast monolithic, layered, microkernel, modular and hybrid strategies for designing operating system
- Illustrate the process for booting an operating system
- Apply tools for monitoring operating system performance
- Design and implement kernel modules for interacting with a Linux kernel

Operating System Services

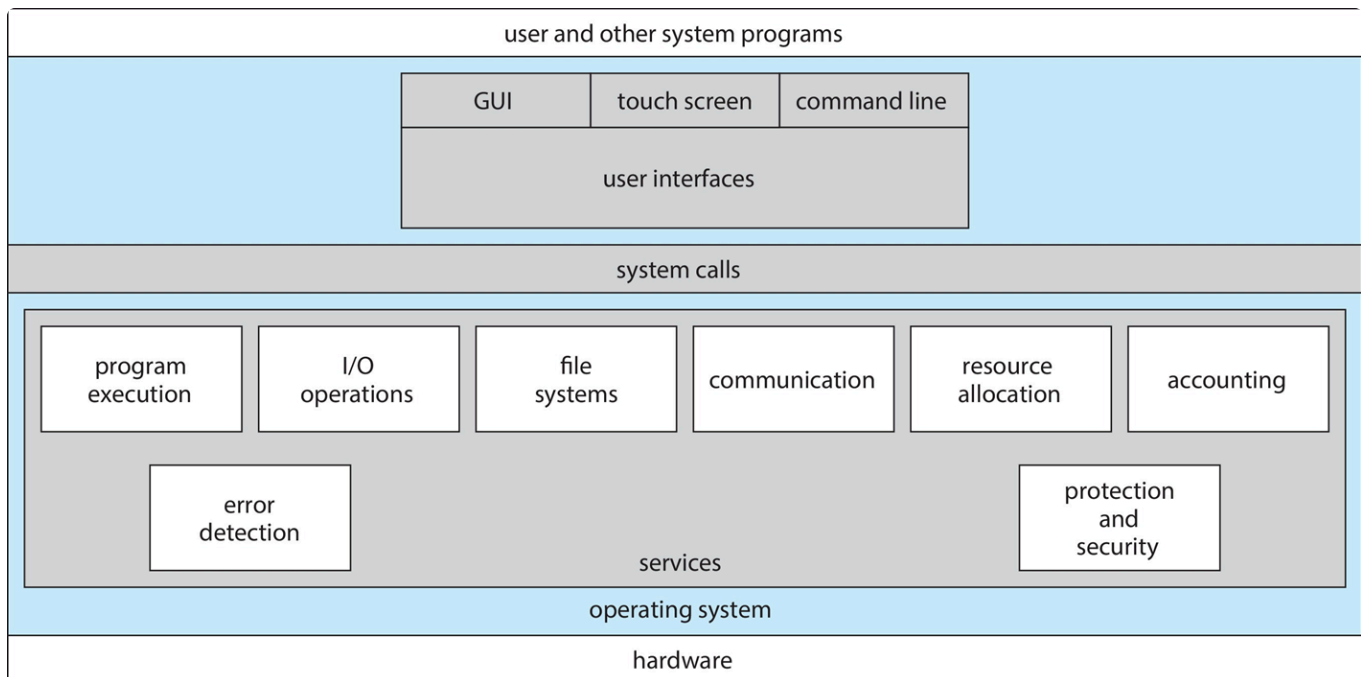
One set of OS services provides functions that are helpful to the user:

- User interface
Almost all operating systems have a UI, varying between Command-Line(CLI), Graphics User Interface(GUI), touch-screen, Batch.
- Program execution
Load a program into memory and to run that program, end execution, maybe indicating error
- I/O operations
- File-system manipulation
Read and write files and directories, create and delete them, search them, list file information and permission management

- Communications
Exchange information on the same computer or between computers over a network(via shared memory or messaging passing)
- Error detection
Be constantly aware of possible errors(CPU, memory hardware, I/O devices or user program)
- Debugging facilities
Enhance the abilities to efficiently use the system

Another set of OS functions exists for ensuring the efficient operation of the system itself via resource sharing.

- Resource allocation
Allocate CPU cycles, main memory, file storage, etc. to multiple users or multiple jobs running concurrently.
- Logging
Keep track of which users use how much and what kinds of computer resources
- Protection and security
Ensure concurrent processes not to interfere with each other in multiuser or networked computer system.
 - Protection: Ensuring all access to system resources is controlled
 - Security: Defending external I/O devices from invalid access attempts.



Command Line Interpreter

- CLI allows direct command entry.

- Sometimes implemented in kernel and sometimes by system program. Multiple flavors implemented, known as **shells**.
- Fetch a command from user and execute it.
- Two types of command: **built-in** and **names of programs**. (If latter, adding new features doesn't require shell modification)

Common Command In Bourne Shell

1. File & Directly Operations

- `ls` - List directory contents
- `cd` - Change directory
- `pwd` - Print working directory
- `cp` - Copy files/directories
- `mv` - Move/rename files/directories
- `rm` - Remove files/directories
- `mkdir` - Create directory
- `rmdir` - Remove empty directory
- `touch` - Create empty file/update timestamps
- `cat` - Concatenate and display files
- `less/more` - Page through file contents
- `head/tail` - Show beginning/end of files
- `find` - Search for files
- `grep` - Search text using patterns

Other types are showed in `CommonCommandsInBShell.txt`.

Common Command In DOS

1. File & Directory Operations

- DIR - List directory contents
- CD or CHDIR - Change directory
- MD or MKDIR - Create directory
- RD or RMDIR - Remove directory
- COPY - Copy files
- XCOPY - Copy files and directories
- MOVE - Move files
- DEL or ERASE - Delete files
- REN or RENAME - Rename files
- TYPE - Display file contents
- ATTRIB - Display/change file attributes
- TREE - Display directory structure
- FC - File compare

Other types are shown in `CommonCommandsInDOS.txt` .

User Operating System Interface--GUI

GUI is **user-friendly desktop** metaphor interface

- Usually mouse, keyboard, and monitor
- **Icons** represent files, programs, actions, etc.
- Various mouse buttons over objects in the interface cause various actions

Many system now include **both CLI and GUI interfaces**.

- Windows is GUI with CLI "command" shell
- Mac OS is "Aqua" GUI interface with Unix kernel and shells available
- Unix and Linux has CLI with optional GUL interfaces(CDE, KDE, GNOME)

Touchscreen Interfaces

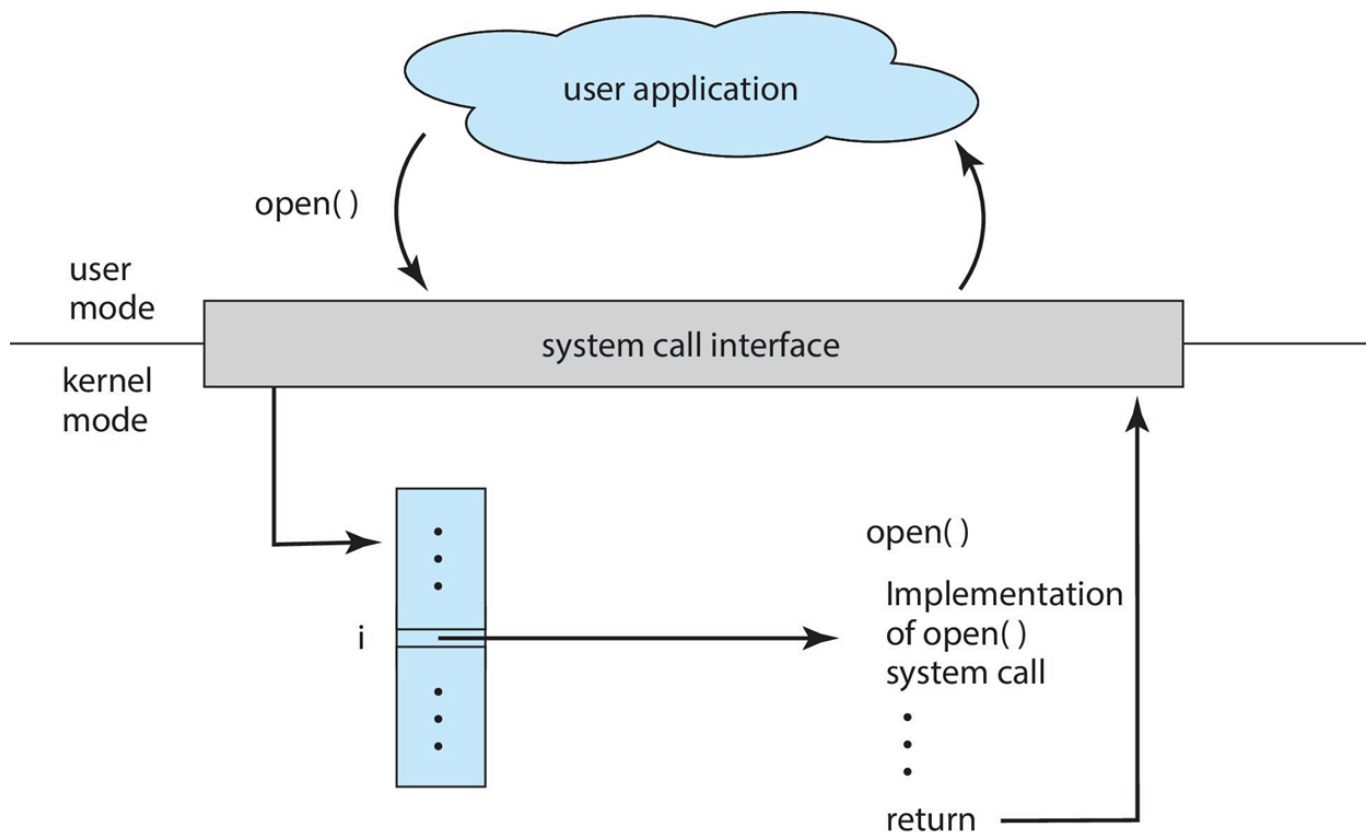
Mouse not possible or not desired. Actions and selections based on gestures. Virtual keyboard for text entry. Voice commands are available.

System Calls

- Programming interface to the services provided by the OS, typically written in a **high-level** language (C or C++)
- Mostly accessed by programs via a high-level **Application Programming Interface(API)** rather than direct system call use.
- Three most common APIs are Win32 API for Windows, POSIX API for POSIX-based systems(including virtually all versions of Unix, Linux and Mac OS), and Java API for the Java virtual machine(JVM)

System Call Implementation

- A number is associated with each system call. **System-call interface** maintains a table indexed according to these numbers.
- The system call interface invokes the intended system call in **OS kernel** and returns **status of the system and any return values**.
- The caller just need to **obey API** and understand **what OS will do as a result call**. Details of how the system call is implemented and OS interface hidden by API, managed by run-time support library



System Call Parameter Passing

More information is required for desired system call. Exact type and amount of information vary according to OS and call.

Three general methods used to pass parameters to the OS:

- Simplest: pass the parameters in registers
- Block passing: When **there is more parameters than registers**, parameters stored in a block(or table) in memory and pass the address of block as a parameter in a register.
- Stack passing: Parameters pushed onto the **stack** by program and popped off the stack by OS.

Block and stack methods **do not limit the number or length of parameters being passed**.

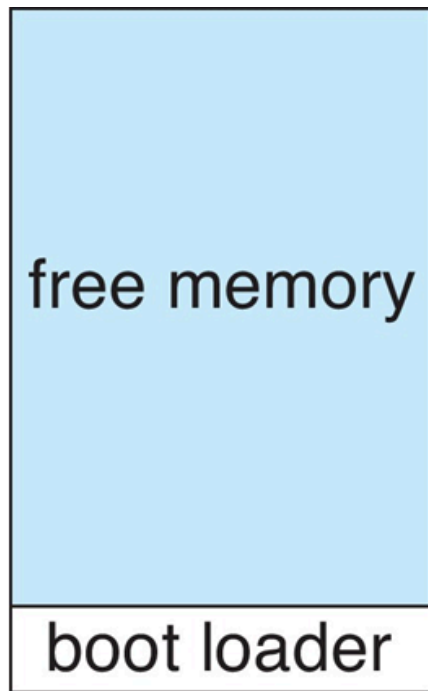
Types of System Calls

- Process control
 - Create process, terminate process
 - End, abort
 - Load, execute
 - Get process attributes, set process attributes
 - Wait for time
 - Wait event, signal event
 - Allocate and free memory
 - Dump memory if error
 - Debugger for determining bugs, single step execution
 - Locks for managing access to shared data between processes
- File management
 - Create file, delete file
 - open, close file
 - read, write, reposition
 - get and set file attributes
- Device management
 - request device, release device
 - read, write, reposition
 - get and set device attributes
 - logically attach or detach devices
- Information maintenance
 - get and set time or date
 - get and set system data

- get and set process, file, or device attributes
- Communications
 - create, delete communication connection
 - send, receive messages if **message passing model** to **host name** or **process name**
 - **Shared-memory** model create and gain access to memory regions
 - Transfer status information
 - Attach and detach remote devices
- Protection
 - Control access to resources
 - Get and set permissions
 - Allow and deny user access

Example: Arduino

- Single-tasking
- No operating system
- Programs loaded via USB into flash memory
- Single memory space
- Boot loader loads program
- Program exit -> shell reloaded



(a)

At system startup



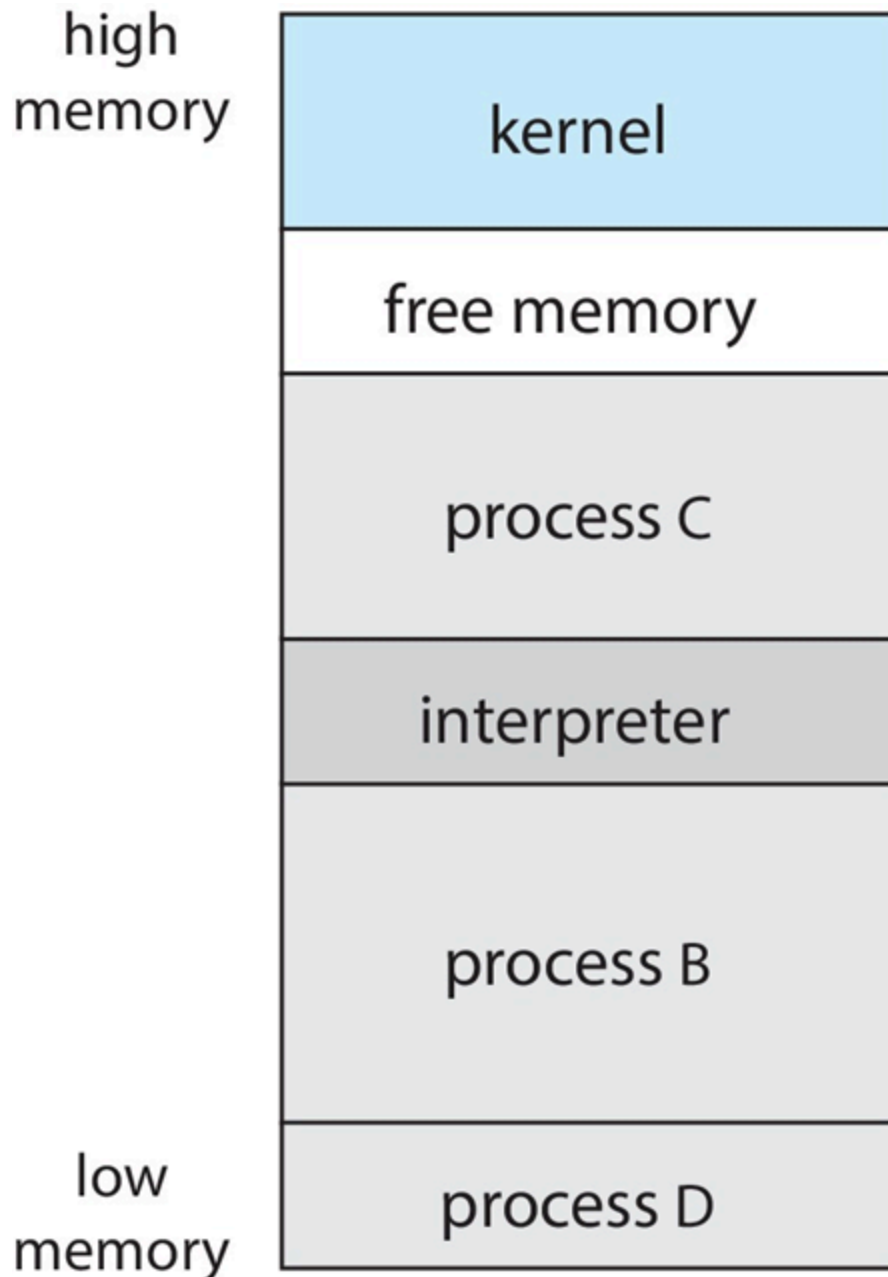
(b)

running a program

Example: FreeBSD

- Unix variant
- Multitasking
- User login -> invoke user's choice of shell
- Shell executes `fork()` system call to create process
 - Executes `exec()` to load program into process
 - Shell waits for process to terminate or continues with user commands

- Process exit with code(=0 -> no error; >0 -> error code)



System Services

Include:

- file manipulation
 - create, delete, copy, rename, print, dump, list and generally manipulate files and directories.
 - File modification
 - Text editors to create and modify files
 - Special commands to search contents of files or perform transformations of the text
- status information sometimes stored in a file
 - Ask the system for info - date, time, etc.

- Provide detailed performance, logging and debugging info.
- Format and print the output to the terminal or other output devices
- Implement a **registry** - used to store and retrieve configuration information.
- programming language support
Compilers, assemblers, debuggers and interpreters sometimes provided.
- program loading and execution
Absolute loaders, relocatable loaders, linkage editors, overlay-loaders, debugging systems for higher-level and machine language.
- communications
 - Provide the mechanism for creating virtual connections among processes, users, and computer systems
 - Allow users to send messages to other users(another screens, browser web pages, etc.)
- background services
 - Launch at boot time, some for startup then terminate whereas some from system boot to shutdown
 - Provide facilities like disk checking, processing scheduling, etc.
 - Run in user context not kernel context
 - Known as services, subsystems, daemons.
- application programs
 - No relation to system
 - Run by users
 - Not typically considered part of OS
 - Launched by command line, mouse click, finger poke

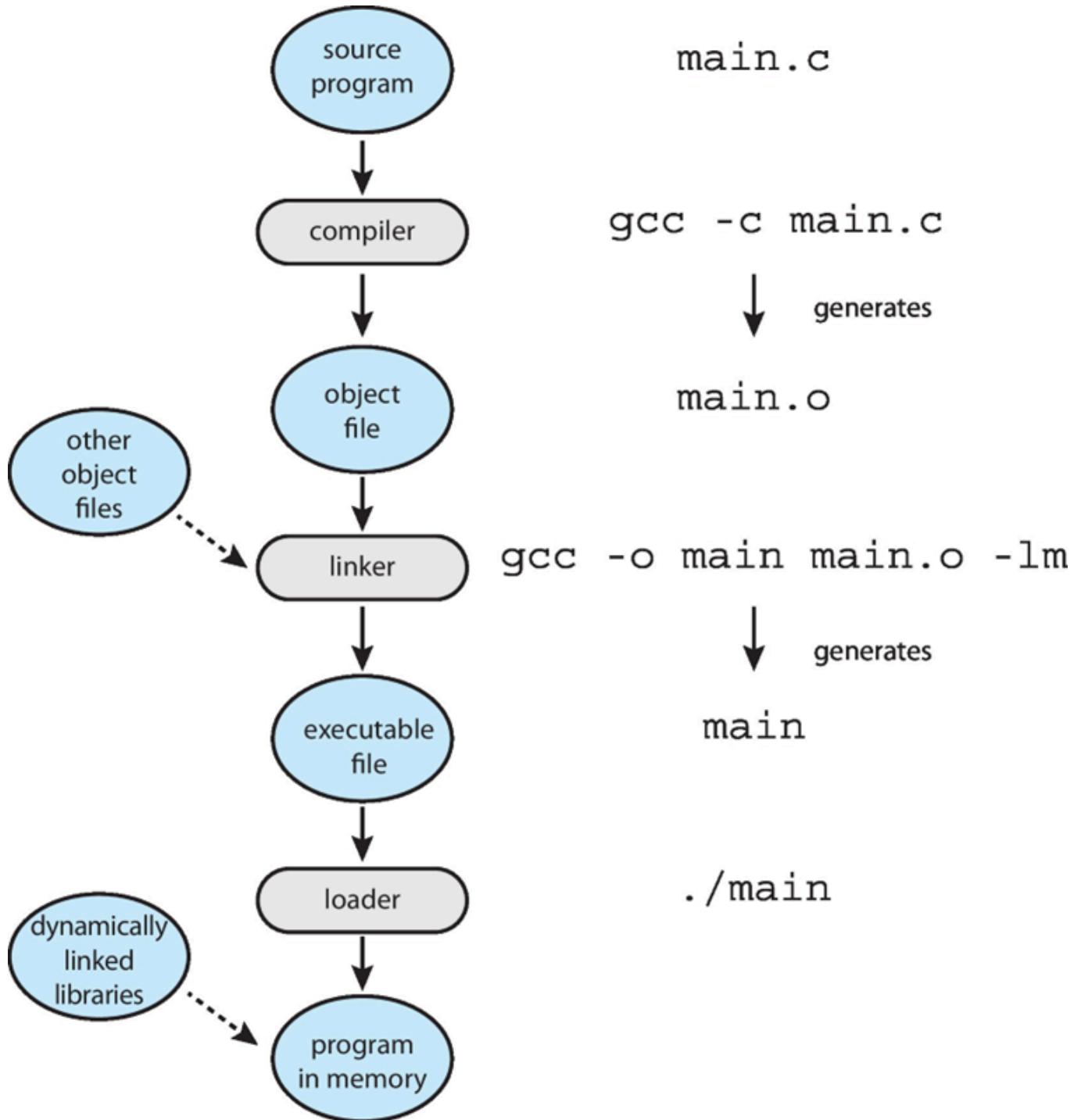
They are provided by **system programs** rather than actual system calls, providing a convenient environment for program development and execution.

Linkers and Loaders

Source code compiled into object files designed to be loaded into any physical memory location(called **relocatable object file**)

Linker combines multiple relocatable object files into single binary **executable** file and brings in libraries. (Modern general purpose system **don't link libraries into executables**, rather, **dynamically linked libraries**, like DLLs in Windows, are loaded as needed, shared by all that use the same version of that same library, only loaded once)

Loader brings the program =resides on secondary storage= as binary executable into memory.



Why Applications are Operating System Specific

- Apps compiled on one system usually **not executable on other operating systems**.
- Each operating system provides its own unique system calls, like own file formats, etc.
- The ways that apps can be multi-operating system:
 - **Written in interpreted language** like Python, Ruby, and **interpreter available** on multiple operating systems.
 - Written in language that **includes a VM containing the running app** (like Java).

- Use **standard language**(like C), compile separately on each operating system to run on each
- **Application Binary Interface(ABI)** is architecture equivalent of API, defines **how** different components of binary code **can interface for a given operating system** on a given architecture, CPU, etc.

Design and Implementation

Internal structure of different OSes can vary widely so the design and implementation of OS is not "solvable". The following are some tips.

- Start the design by defining goals and specifications
- The design affected by choice of hardware, type of system
- **User** goals and **System** goals
 - User goals: OS should be convenient to use and easy to learn, reliable, safe, fast
 - System goals: OS should be easy to design, implement and maintain, and flexible, reliable, error-free and efficient.
- It is mainly the task of **software engineering**.

Policy and Mechanism

- Policy: **What** needs to be done. E.g., interrupt after every 100 seconds
- Mechanism: **How** to do something. E.g., use timer to do periodic interrupt.

The important principle is **to separate policy from mechanism**, allowing maximum flexibility if policy decision are to be change later.(like change 100 seconds to 200 seconds)

Implementation

- Much variation.
Early OSes in assembly language, then system programming languages like Algol, PL/1, now C, C++
- Actually **usually a mix of languages**
Lowest levels in assembly, main body in C and system programs in C, C++, scripting languages like PERL, Python, shell scripts
- More high-level language **easier to port to other hardware** but **slower**.
- **Emulation** can allow an OS to run on non-native hardware.

Operating System Structure

General-purpose OS is very large program and exists various ways to structure ones

- Simple structure - MS-DOS
- More complex - Unix
- Layered - an abstraction
- Microkernel - Mach

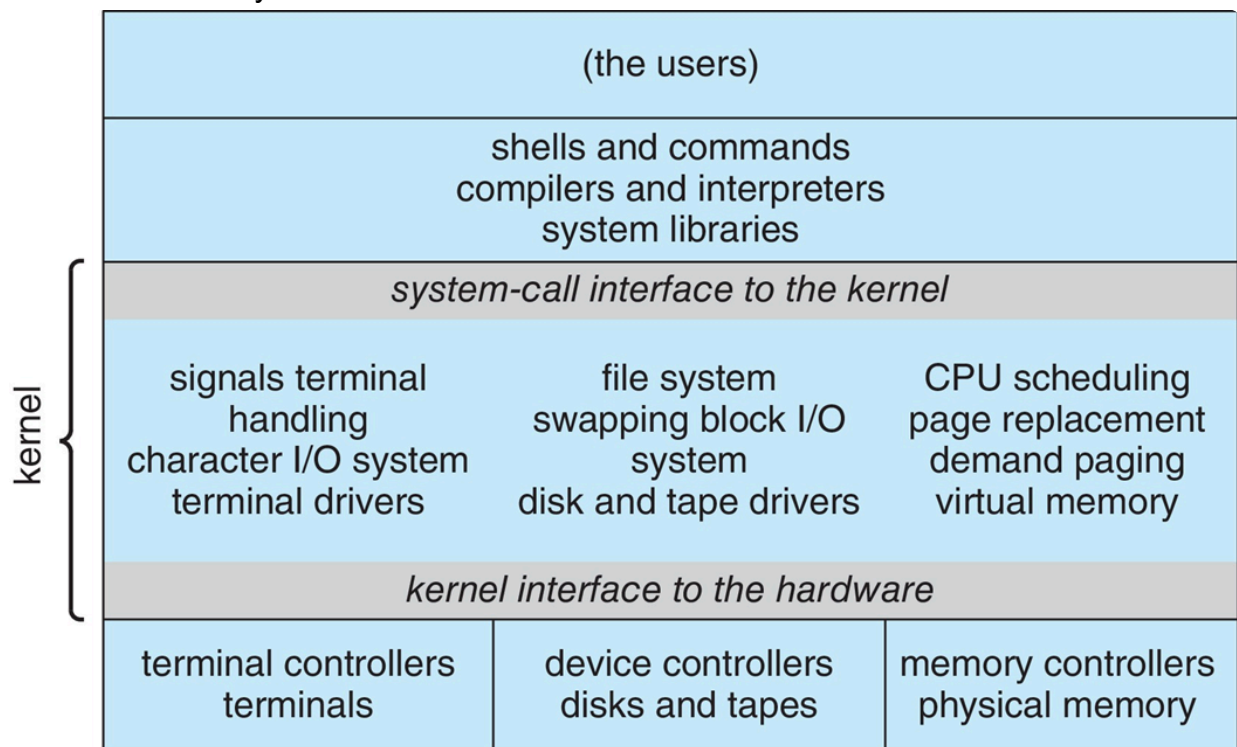
Monolithic Structure - Original Unix

Original Unix is **limited by hardware functionality** and have a low structuring. It is **beyond simple but not fully layered**.

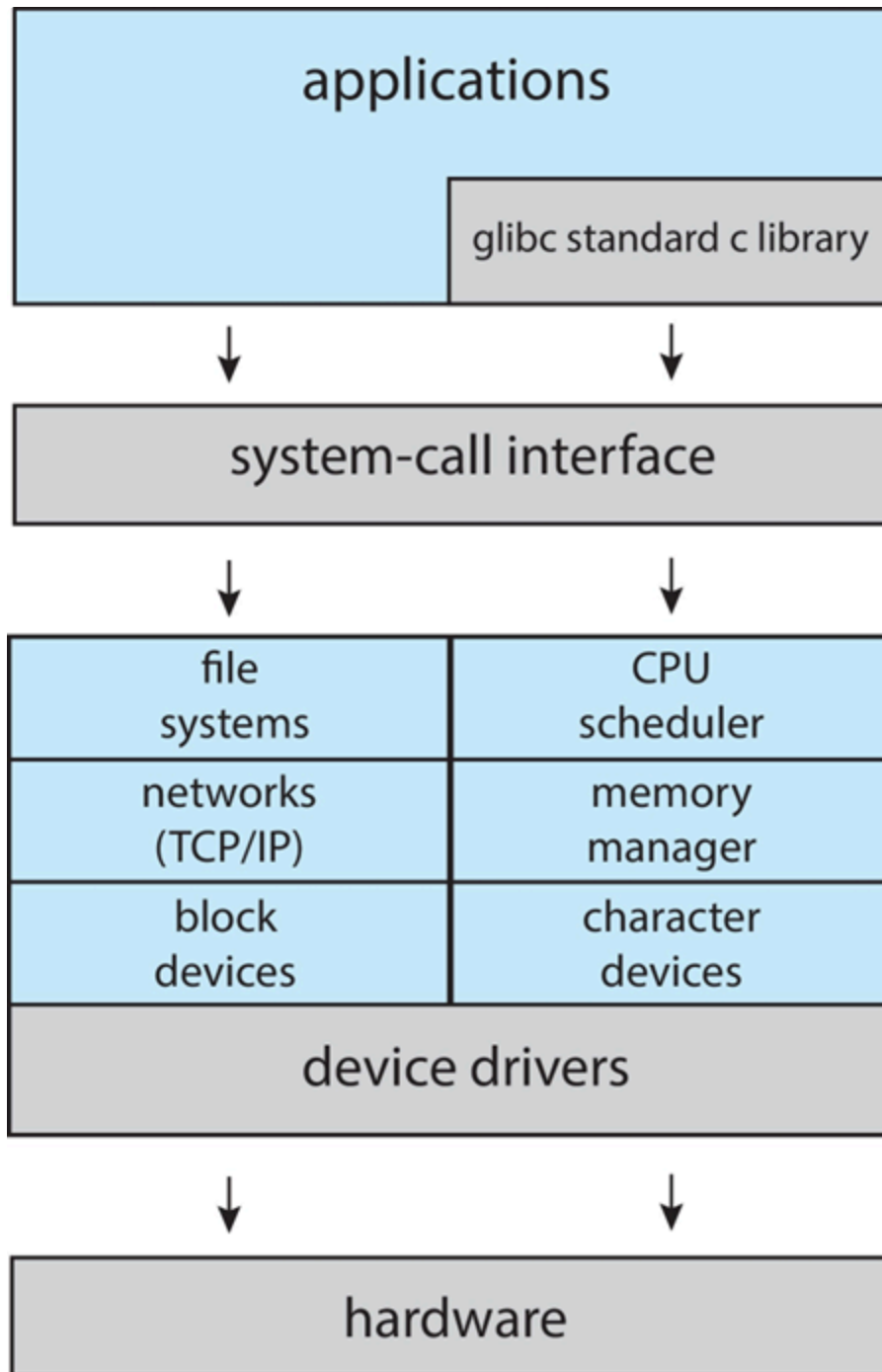
The Unix OS consists of two separable parts:

- System Programs
- Kernel
 - Consists of everything below the system-call interface and above the physical hardware
 - Provides the file system, CPU scheduling, memory management and other operating-system functions, a large number of functions for one level.

Traditional Unix system structure:



Linux system structure:



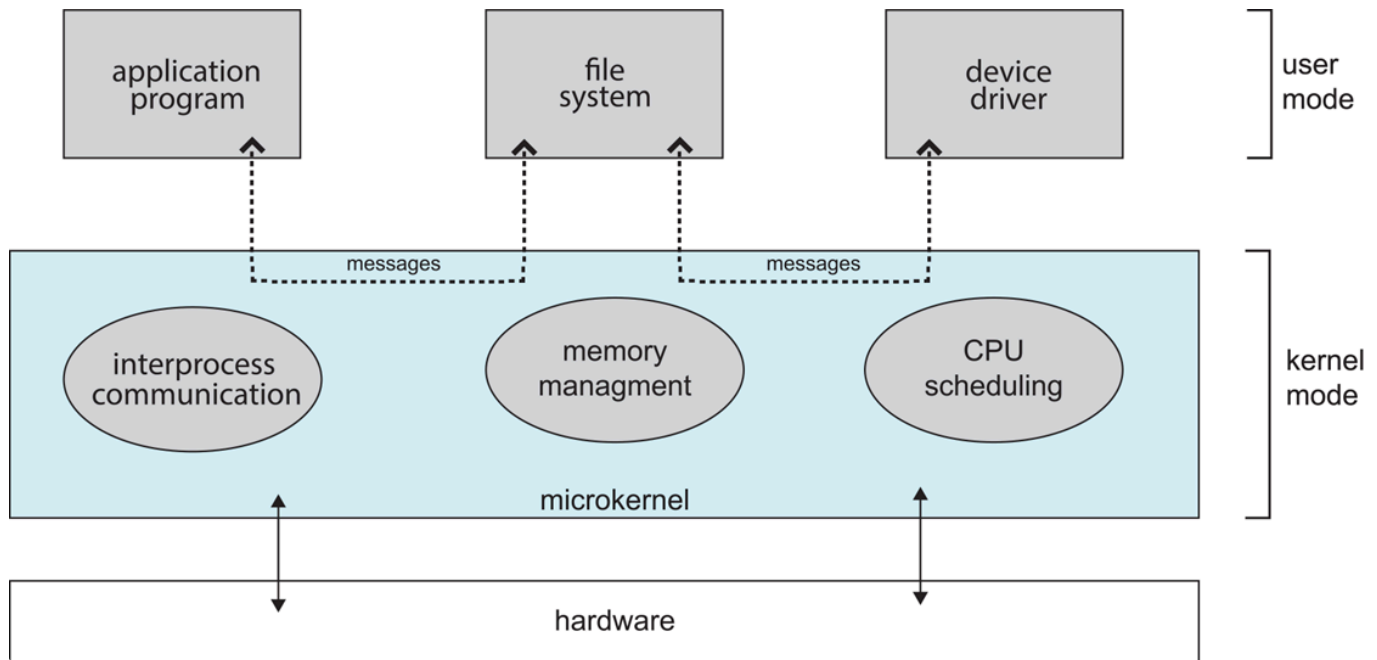
Layered Approach

- The operating system is divided into a number of layers, each built on top of lower layers. The bottom layer(layer 0) is the hardware and the highest is the user interface.
- With **modularity**, layers are selected such that each uses functions and services of **only lower-level layers**.

Microkernels

- Moves as much functions **from the kernel into user space**.

- **Mach** is an example of **microkernel** and Mac OS X kernel (Darwin) partly based on Mach
- Communication takes place between user modules using **message passing**
- Benefits:
 - Easier to extend a microkernel
 - Easier to port the operating system to new architectures
 - More reliable
 - More secure
- Detriments
 - **Performance overhead** of user space to kernel space communication



Modules

Many modern operating systems implement **loadable kernel modules** (LKMs)

- Uses **object-oriented** approach
- Each core component is **separate**
- Each talks to the others over known interfaces
- Each is loadable as needed within the kernel

Overall it is similar to layers but with more flexible.(Linux, Solaris, etc.)

Hybrid Systems

Most modern system are not one pure model. It combines multiple approaches to address performance, security, usability needs.(Linux, Window, Mac OS X, etc.)

Building and Booting an Operating System

Operating systems generally designed to run on a class of systems with variety of peripherals.

Commonly, OS already installed on purchased computer. But can build and install some other operating systems.

To generate an OS from scratch:

- Write the OS source code
- Configure the OS for the system on which it will run(hardware)
- Compile the OS
- Install the OS
- Boot the computer and its new OS

For example, building and booting Linux.

Building and Booting Linux

- Download Linux source code (www.kernel.org)
- Configure kernel via `make menuconfig`
- Compile the kernel using `make`
 - Produces `vmlinuz`, the kernel image
 - compile kernel modules via `make modules`
 - Install kernel modules into `vmlinuz` via `make modules_install`
 - Install new kernel on the system via `make install`

System Boot

- Power initialized on system, execution starts at a fixed memory location.
- BIOS(basic input/output system), the hardware stored in ROM or EEPROM, locates the bootstrap loader, loads it into memory and starts it.
 - Boot loaders frequently allow various boot states, such as single user mode.
 - Sometimes two steps. ROM code loads boot block at fixed location. Then boot block loads bootstrap loader from disk.
 - Modern system replace BIOS with Unified Extensible Firmware Interface(UEFI).
 - Common bootstrap loader, GRUB, allows selection of kernel form multiple disks, versions, kernel options.
- Kernel is loaded by bootstrap loader.
- System is then running.

Operating-System Debugging

Debugging is finding and fixing errors or bugs and tuning performance.

- OS generate **log files containing error information**.
 - Applications failure can generate **core dump** file containing process memory.
 - OS failure can generate **crash dump** file containing kernel memory.
- Performance tuning can optimize system performance
 - Sometimes using **trace listings** of activities, recorded for analysis.
 - **Profiling** is periodic sampling of instruction pointer to **look for statistical trends**.

In fact, debugging is twice as hard as writing the code in the first place.

Performance Tuning

- Improve performance by removing bottlenecks
- OS must provide means of computing and displaying measures of system behavior. ("top" program of Linux, task manager of Windows)

Tracing

Collects data for a specific event.(e.g., steps involved in a system call invocation)

Tools include:

- strace - trace system calls invoked by a process
- gdb - source-level debugger
- perf - collection of Linux performance tools
- tcpdump - collects network packets

BCC

Debugging interactions between user-level and kernel code nearly impossible without toolset that understands both and an instrument their actions.

BCC (BPF compiler collection) is a rich toolkit providing tracing features of Linux. There are also many other tools.